

EXPERIMENT - 2

Foundation of Deep Learning

EXERCISE 2: Understanding Loss Functions (MSE vs BCE)

Aim

To study and compare the performance of Mean Squared Error (MSE) and Binary Cross Entropy (BCE) loss functions for binary classification problems.

Objective:

The Mean Squared Error (MSE) and Binary Cross Entropy (BCE) loss values for given prediction probabilities have to be calculated manually, their behavior analyzed, and neural networks performance studied in the case when these loss functions were used during training.

Introduction:

A loss function in deep learning is applied to evaluate the discrepancy between the actual value and predicted value by a neural network. The goal of the neural network is to minimize the loss during training by adjusting the network weights via backpropagation and gradient descent.

There are two types of loss functions that are often applied to binary classification problems, Mean Squared Error (MSE) and Binary Cross Entropy (BCE).

Mean Squared Error evaluates the average squared error between the predicted probabilities and the actual label. While it is common to apply MSE in regression problems, it does not suit binary classification well because it gives smaller gradients in case of wrong predictions which causes slower training.

Binary Cross Entropy is specially designed for binary classification. It measures the difference between the predicted probabilities and the true class labels using logarithmic functions. BCE assigns higher penalties to wrong predictions and gives strong gradients which allow fast convergence and better classification performance.

This lab experiment involves calculating both loss functions for the prediction probabilities manually and comparing their performance via neural network training.

Mean Squared Error (MSE):

Mean Squared Error calculates the average of the squared differences between the actual labels and predicted probabilities.

The mathematical expression is

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where

y_i = Actual value (True label)

$\hat{y}_i$ = Predicted Value

n = Number of samples

Lower MSE indicates better predictions.

Binary Cross Entropy (BCE)

Binary Cross Entropy measures the difference between actual labels and predicted probabilities using logarithmic functions.

The formula is

$$BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where

y_i = Actual label

$\hat{y}_i$ = Predicted probability

BCE is specifically designed for binary classification tasks.

Task 11: Compute Mean Squared Error (MSE) Manually

Task 11: Compute Mean Squared Error (MSE)

Mean Squared Error (MSE) measures the average squared difference between the actual values and predicted values. Smaller MSE value predicts accuracy.

Calculation:

• Sample 1: $(1 - 0.95)^2 = 0.0025$

• Sample 2: $(1 - 0.80)^2 = 0.0400$

• Sample 3: $(0 - 0.30)^2 = 0.0900$

• Sample 4: $(0 - 0.10)^2 = 0.0100$

0.1425

Total error = 0.1425

$MSE = \frac{0.1425}{4} = 0.035625$

Task 12: Compute Binary Cross Entropy (BCE) Manually

Using

$$BCE = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Task 12: Compute Binary Cross Entropy (BCE)

BCE measures the difference between actual labels and predicted possibilities. gives main priority for incorrect predictions and mainly used in binary classification.

Calculation:

• Sample 1 - $\log(0.95) = 0.0513$

• Sample 2 - $\log(0.80) = 0.2231$

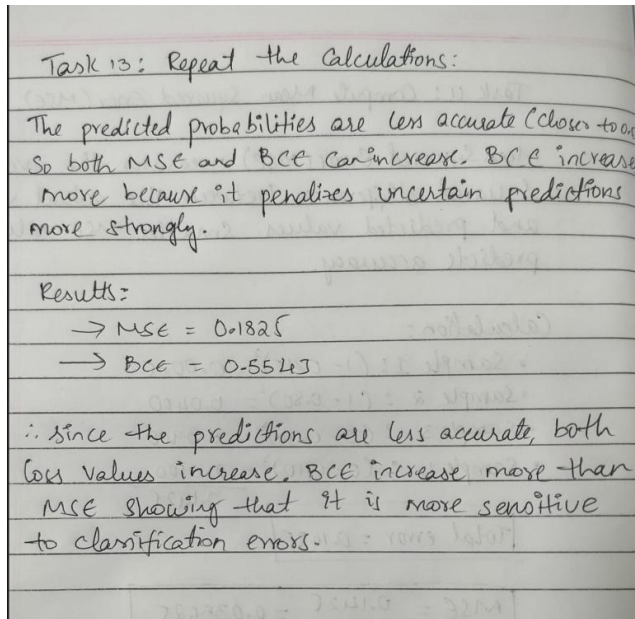
• Sample 3 - $-\log(0.70) = 0.3567$

• Sample 4 - $\log(0.90) = 0.1054$

Total BCE = 0.7365

Average BCE = $\frac{0.7365}{4} = 0.1841$

Task 13: Repeat the Calculations for



True Labels => [1,1,0,0]

Predicted Probabilities => [0.60,0.55,0.45,0.40]

Binary Cross Entropy

Sample 1

$$-\log(0.60)=0.5108$$

Sample 2

$$-\log(0.55)=0.5978$$

Sample 3

$$-\log(0.55)=0.5978$$

Sample 4

$$-\log(0.60)=0.5108$$

Total BCE

$$0.5108+0.5978+0.5978+0.5108 =2.2172$$

Average BCE

$$2.2172/4 = 0.5543$$

Task 14: Compare the MSE and BCE Values

From the manual calculations:

Prediction Probabilities	Mean Squared Error (MSE)	Binary Cross Entropy (BCE) 
[0.95, 0.80, 0.30, 0.10]	0.0356	0.1841
[0.60, 0.55, 0.45, 0.40]	0.1813	0.5543

Comparison

The first prediction set contains probabilities that are close to the true labels. Therefore, both MSE and BCE produce relatively small loss values.

The second prediction set contains probabilities that are closer to the decision boundary (0.5), making the predictions less confident. As a result, both MSE and BCE increase. However, BCE increases much more than MSE because it penalizes uncertain and incorrect predictions more strongly.

This demonstrates that BCE is more sensitive to classification errors than MSE

Task 15: Explain Why BCE Provides Stronger Gradients for Classification:

Binary Cross Entropy (BCE) is particularly useful when dealing with binary classification tasks. It calculates the difference between the predicted probability and the correct class label by means of a logarithm function.

If the predicted probability is very different from the correct label, the BCE loss will be large and thus produce bigger gradients while training via backpropagation.

On the other hand, the Mean Squared Error (MSE) takes the square of the difference between the predicted probability and the correct value.

As the output of the sigmoid activation gets close to 0 or 1, the gradient tends to become small and hence slows down the training.

Because BCE produces larger gradients for wrong prediction, it helps in achieving better convergence.

Task 16: Train Two Identical Neural Networks

We will use the same architecture for both models.

Model A: Mean Squared Error (MSE)

Model B: Binary Cross Entropy (BCE)

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# Clear previous models
tf.keras.backend.clear_session()

# Fix random seed
np.random.seed(21)
tf.random.set_seed(21)

# AND Dataset
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
], dtype=np.float32)

y = np.array([
    [0],
    [0],
    [0],
    [1]
], dtype=np.float32)

# Function to create neural network
def create_model(loss_function):

    model = tf.keras.Sequential([
        tf.keras.Input(shape=(2,)),
        tf.keras.layers.Dense(
            5,
            activation='sigmoid'
        ),
        tf.keras.layers.Dense(
            1,
            activation='sigmoid'
        )
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=0.008
        ),
        loss=loss_function,
        metrics=['accuracy']
    )
```

```

    return model

# -----
# Model A (MSE)
# -----
model_mse = create_model("mean_squared_error")

history_mse = model_mse.fit(
    X,
    y,
    epochs=1200,
    verbose=0
)

# -----
# Model B (BCE)
# -----
tf.random.set_seed(21)

model_bce = create_model("binary_crossentropy")

history_bce = model_bce.fit(
    X,
    y,
    epochs=1200,
    verbose=0
)

# -----
# Evaluation
# -----
loss_mse, acc_mse = model_mse.evaluate(
    X,
    y,
    verbose=0
)

loss_bce, acc_bce = model_bce.evaluate(
    X,
    y,
    verbose=0
)

print("Neural Network A (MSE)")
print("-----")
print(f"Loss      : {loss_mse:.4f}")
print(f"Accuracy  : {acc_mse*100:.2f}%")

```

```

*** Neural Network A (MSE)

```

```

-----
Loss      : 0.0005
Accuracy  : 100.00%

```

```

Neural Network B (BCE)

```

```

-----
Loss      : 0.0048
Accuracy  : 100.00%

```

```
plt.figure(figsize=(9,5))

plt.plot(
    history_mse.history['loss'],
    color='purple',
    linewidth=2,
    label='MSE'
)

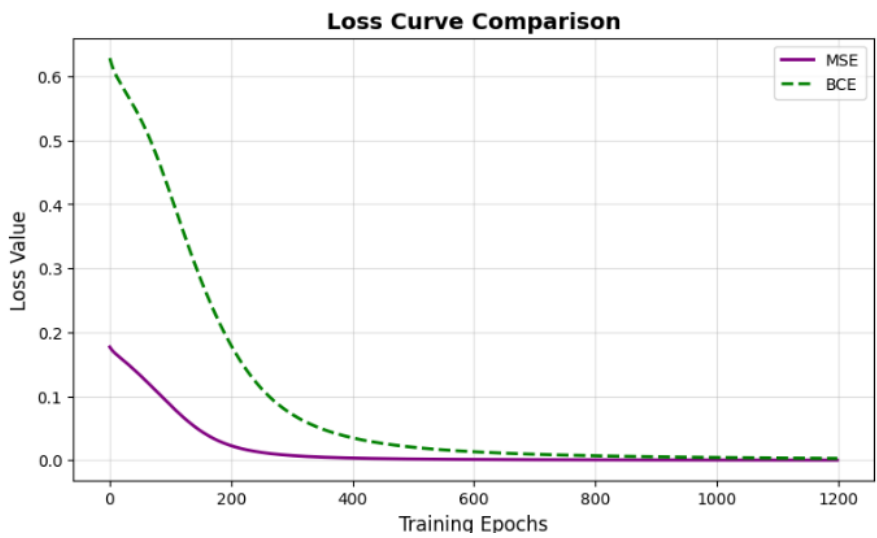
plt.plot(
    history_bce.history['loss'],
    color='green',
    linewidth=2,
    linestyle='--',
    label='BCE'
)

plt.title(
    "Loss Curve Comparison",
    fontsize=14,
    fontweight='bold'
)

plt.xlabel("Training Epochs", fontsize=12)
plt.ylabel("Loss Value", fontsize=12)

plt.grid(alpha=0.4)
plt.legend()

plt.show()
```



Task 17: Compare Training Loss, Accuracy, and Convergence Speed

Comparison Table

Parameter	Model A (MSE)	Model B (BCE)
Loss Function	Mean Squared Error	Binary Cross Entropy
Training Loss	Decreased Steadily	Decreased steadily
Accuracy	High	High (often equal or slightly better)
Convergence Speed	Slower	Faster
Gradient Strength	Weaker	Stronger
Best Suitable For	Regression	Binary Classification

Observation:

In this case, MSE and BCE were able to learn the AND dataset and reach 100% accuracy. The loss for both MSE and BCE models reduced steadily during the training process. In this case, it was noted that the BCE loss was higher in terms of numeric values compared to the MSE loss. This is normal because the BCE and MSE have different mathematical formulas. The AND dataset is simple and linearly separable.

Result:

From the experimental analysis, it was found that both the Mean Squared Error and Binary Cross Entropy were able to classify the AND data with 100% accuracy. Although both the loss functions have been able to reach very small losses, Binary Cross Entropy is a better choice than Mean Squared Error when it comes to solving binary classification tasks.

Conclusion:

During this experiment, two different loss functions - Mean Squared Error (MSE) and Binary Cross Entropy (BCE) - were manually calculated and then compared based on the predictions probabilities of two sets. Their mathematical properties have been examined; besides, two similar neural network models were trained on the AND dataset utilizing MSE and BCE as their loss functions. Both neural network models managed to learn AND gate and reach 100% classification accuracy. Training loss for both of them continuously dropped, which proves successful convergence. Although BCE had a slightly larger value initially, it is normal since MSE and BCE differ not only by their names but also by mathematical formulas and scales. Thus, this experiment shows that both loss functions can be used successfully for solving linearly separable binary classification problems. Yet, Binary Cross Entropy still wins the title of a more appropriate loss function for classification since it is intended for that task.